رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

# Data Analysis to Answer Questions

Session 4

# Agenda

- Recap
- Explore Data Results Using Pandas, NumPy, Seaborn, & Matplotlib in Python

# Introduction to NumPy

- NumPy is short for Numerical Python and, as the name indicates, it deals with everything related to Scientific Computing. The basic object in NumPy is the *ndarray* which is also a short for n-dimentional array and in a mathematical context it means multi-dimentional array.

- Any mathematical operation such as differentiation, optimization, solving equations simultaneously will need to be defined in a matrix format to be done properly and easily and that was the purpose of programming languages like *Matlab*.

- Unlike any other python object, ndarray has some interesting aspects that ease any mathematical computation.

# Introduction to NumPy

- NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ndarray will create a new array and delete the original.
- The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory.
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python’s built-in sequences.
- For the sake of Data Analytics there will not be a lot of mathematical computation problems but later on when we will start working with data in tables. You will figure out that any table is more or less a 2 dimensional array and that's why it's essiential to know a bit about array that will convert in future lessons to tables of data.

# NumPy

**What is an array?**

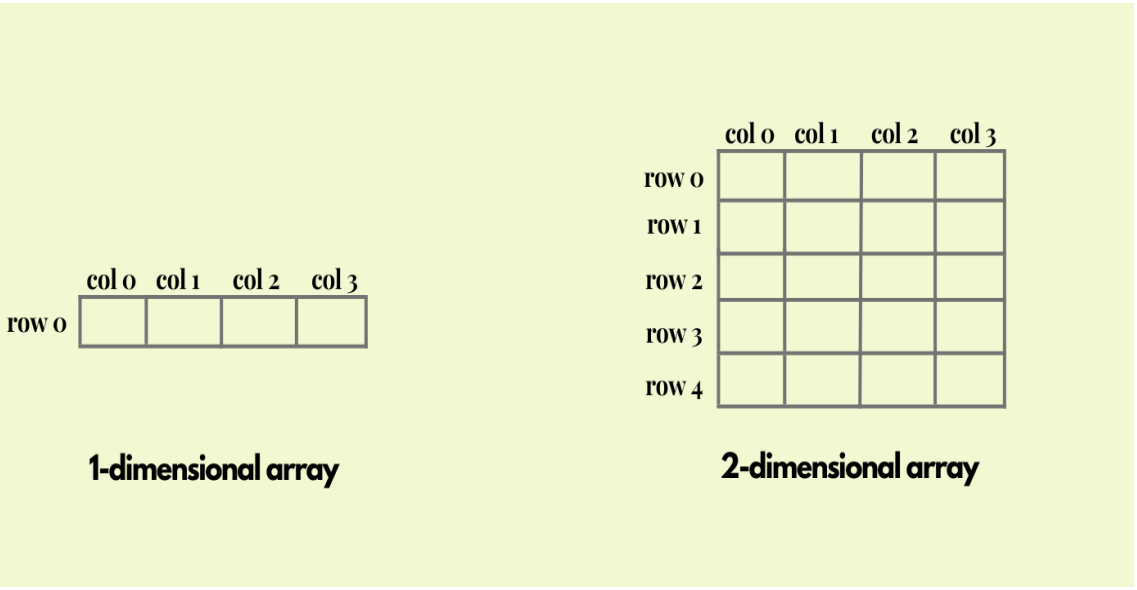
Array is a mathematical object that is defined to hold some numbers organized in rows and columns. The structure of the array should allow selecting (indexing) any of the inner items. Later on we will see how to do this in code.

**Example:**

```
import numpy as np
# Let's create a 1-d array
arr1d = np.array([[1, 2, 3]])
# print its content
print(arr1d)
# print array type
print(type(arr1d))
```

Output

```
[[1 2 3]]
<class 'numpy.ndarray'>
```



# NumPy

Array Shape is the most important aspect to take care of when dealing with array and array-maths in general.

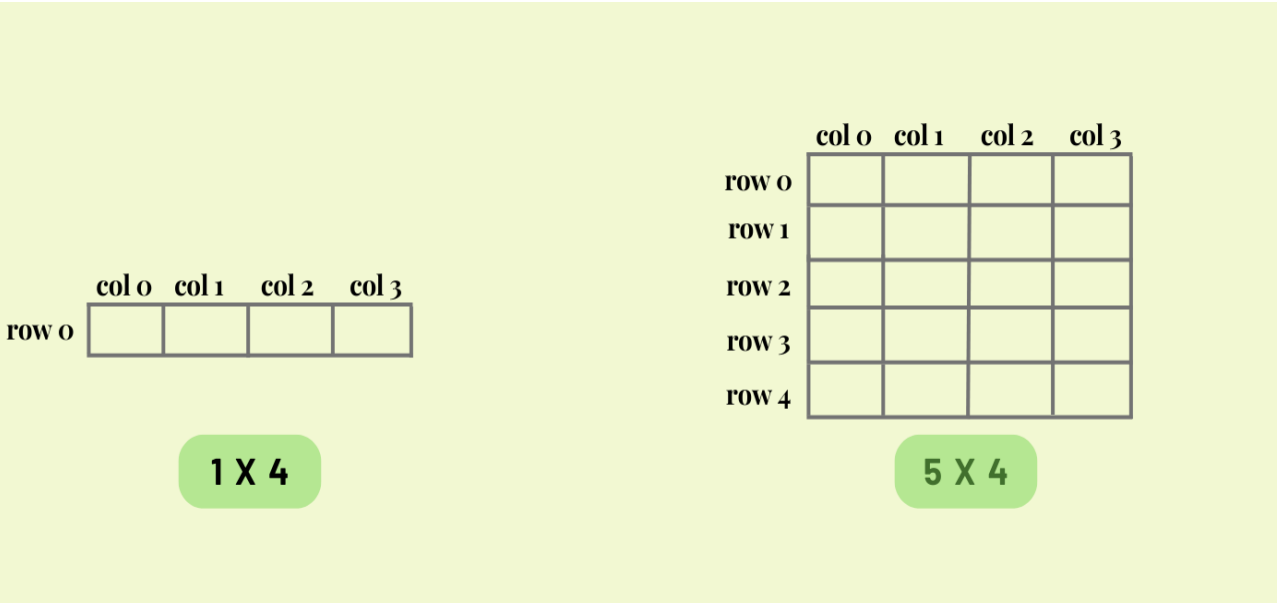
**It’s Simply Shape = N rows x N Columns**

It's a major info to know espcially when dealing with multiplication in arrays. Lets see it in a more visual way.

**Example:**

```
import numpy as np
# Create a 2-d array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
# Print its content
print(arr2d)
# Print array type
print(type(arr2d))
```

```
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>
```



# NumPy

In some cases, we will need to create some special types of arrays such as array of zeros, identity array, or array of ones, etc..

Lets see some examples that could be implanted using NumPy

- **np.zeros()** : creating array of all zeros.
- **np.ones()** : creating array of all ones.
- **np.empty()** : creating array of random values.
- **np.full()** : creating array full of the same number.
- **np.eye()** : creating an identity array. and much more!

```
zeros = np.zeros((2,2))   # Create an array of all zeros (2 rows, and 2 columns)
print(zeros)
```

**Output**

| | |
|---|---|
| 0 | 0 |
| 0 | 0 |

```
ones = np.ones((5,2))    # Create an array of all ones (5 rows and 2 columns)
print(ones)
```

**Output**

[
[1,1]
[1,1]
[1,1]
[1,1]
[1,1]
]

```
full = np.full((5,4), -9)  # Create a constant array
print(full)
```

**Output**

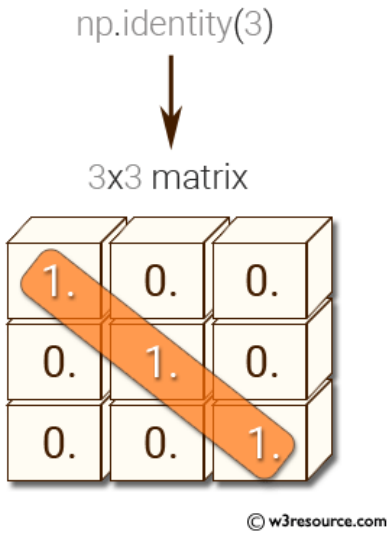
```
[[-9 -9 -9 -9]
 [-9 -9 -9 -9]
 [-9 -9 -9 -9]
 [-9 -9 -9 -9]
 [-9 -9 -9 -9]]
```

# NumPy

- To create it: **np.eye(the number of ones)**

```
eye = np.eye(5)          # Create a 2x2 identity matrix
print(eye)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```



# NumPy



## Array Slicing And Indexing:

```
# Create the following rank with shape (3, 4)
myarr = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
print(myarr)
```

Suppose that we want to select the second row of `myarr`?

```
# Lets type the syntax for selecting the second row.
row_r2 = myarr[1:2, :]
print(row_r2)
```

Now, lets try selecting the second column with the same manner.

```
col_c2 = myarr[:, 1:2]
print(col_c2)
```

Now, lets select the slice that come from the first two rows and two columns

```
myarr_slice = myarr[:2, :2]
print(myarr_slice)
```

# NumPy

**Integer Array Indexing**

- What if we want to select some specific elements in the array?
  - ➢ We should select this based on the mathematical indexing and, for sure, with applying the zero-indexing.

| | col 0 | col 1 | col 2 |
|---|---|---|---|
| row 0 | (0, 0) | (0, 1) | (0, 2) |
| row 1 | (1, 0) | (1, 1) | (1, 2) |
| row 2 | (2, 0) | (2, 1) | (2, 2) |

# NumPy



**Example:**

```
# Lets define a new array
newarr = np.array([[1,2], [3, 4], [5, 6]])
# Print it
print(newarr)
```

```
[[1 2]
 [3 4]
 [5 6]]
```

```
#First part calls the rows and the second one calls the columns
print(newarr[[0, 1, 2], [0, 1, 0]])  # Prints "[1 4 5]"
#mains print element od [0,0], [1,1] and [2,0]
```

# NumPy



**Data Types in NumPy**

NumPy has some data types that could be referred with one character, like i for integers, u for unsigned integers etc.

Below is a list of all data types in NumPy and the characters used to represent them.

| Character | Data Type |
|---|---|
| i | Integer |
| b | Boolean |
| u | Unsigned Integer |
| f | Float |
| c | Complex Float |
| m | Timedelta |
| M | Datetime |
| O | Object |
| S | String |
| U | Unicode String |
| V | Fixed chunk of memory for other type (void) |

In general, we will not use all of them.
Only the famous ones are heavily used such as integer, float, string.
Now, lets see how to check a datatype for a NumPy array.

# NumPy



## Example 1

```
[13] x = np.array([1, 2])
     print(x.dtype)
```

```
int64
```

## Example 2

```
y = np.array([1.0, 2.0])
print(y.dtype)
```

```
float64
```

# Pandas

**Pandas** is an open-source Python library that stands out as one of the most widely used tools for data science, data analysis, and machine learning tasks. Built on top of NumPy, which provides support for multi-dimensional arrays, Pandas offers powerful data structures and functions that facilitate easy data manipulation.

# Pandas

**Key Features of Pandas**

Pandas excels in simplifying a variety of time-consuming and repetitive tasks associated with data handling. Here are some of its key functionalities:

- **Data Cleansing**: Easily handle missing or inconsistent data.
- **Data Filling**: Fill missing values using different strategies (e.g., forward fill, backward fill).
- **Data Normalization**: Standardize data to a common format, which is crucial for analysis.
- **Merges and Joins**: Combine multiple datasets based on common keys or indices.
- **Data Visualization**: Integrate with libraries like Matplotlib and Seaborn for visual representation of data.
- **Statistical Analysis**: Perform descriptive statistics and group analyses with ease.
- **Data Inspection**: Quickly view and explore data using built-in functions.
- **Loading and Saving Data**: Read from and write to various file formats (CSV, Excel, SQL databases, etc.).

# Pandas

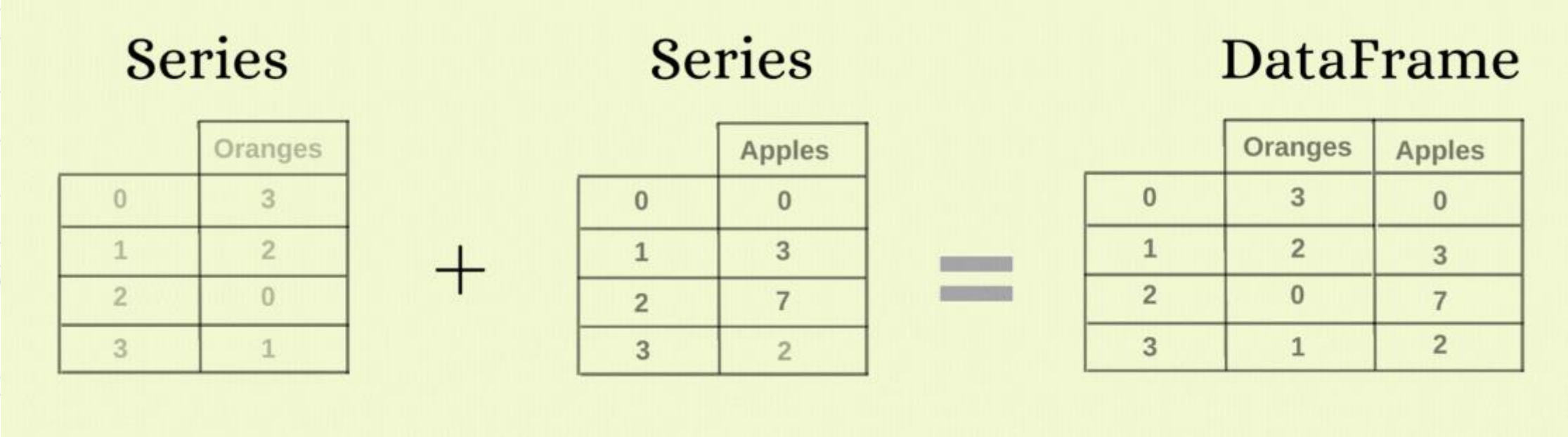
**Why Use Pandas?**

Pandas has become a favorite among data scientists and analysts due to its versatility and the efficiency it brings to data manipulation. Its ability to handle large datasets with ease and perform complex operations quickly makes it a cornerstone of the data analysis workflow in Python.

With its rich functionality and seamless integration with other libraries in the Python ecosystem, Pandas is consistently recognized as one of the best tools for data analysis and manipulation, making it a staple in the toolkit of leading data professionals.

# Pandas

- The power of Pandas lies in its fundamental data structures: the **DataFrame** and the **Series**. These structures are designed to facilitate easy transformation and manipulation of data, making them essential tools for data analysis.



# Pandas



- **Creating a DataFrame**
- DataFrames can be created of various datatypes, but the most straight forward approach is to create a DataFrame from a Dict where the keys are the column names, and the values are lists.

```
import pandas as pd

# Sample data as a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

# Creating a DataFrame from the dictionary
df = pd.DataFrame(data)

# Display the DataFrame
print(df)
```

# Pandas

- To use pandas properly, you need to do two things:
- Install pandas
- Import the package into your current environment.
- Every package in Python is just some code files that are published online for anyone to use or edit. Importing a package means that you are bringing these functions and codes into your current environment which will allow you to use any of these code functions, methods, or attributes. To import pandas, you need to use the following line.

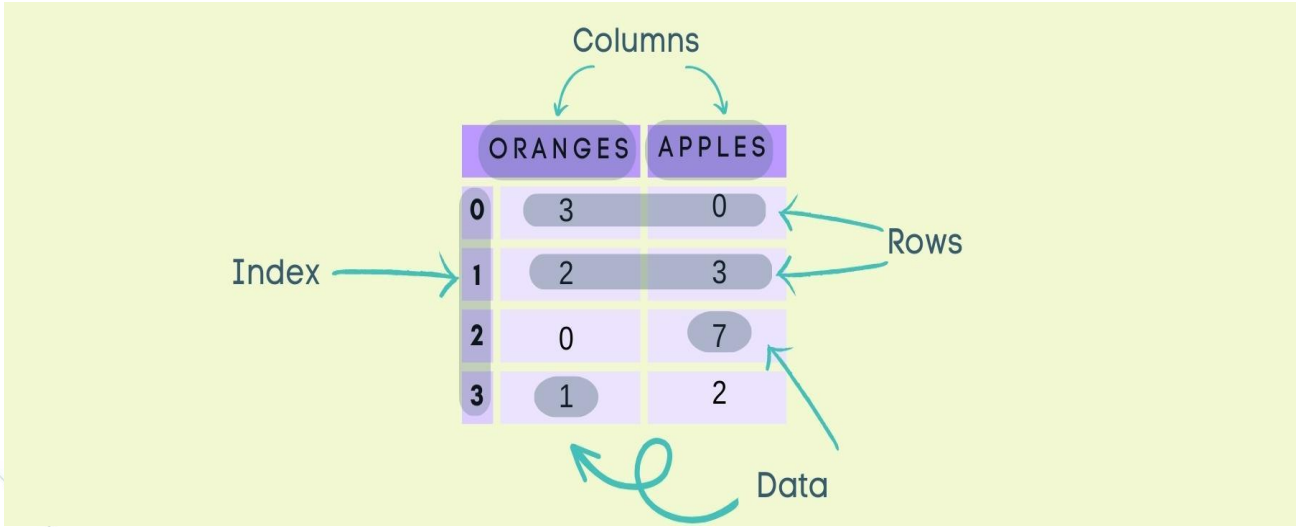
```
# import statment
import pandas as pd

# lets create our dataframe from fruits dct
fruits_df = pd.DataFrame(fruits_dct)

# lets print it
fruits_df
```

| | Oranges | Apples |
|---|---|---|
| 0 | 3 | 0 |
| 1 | 2 | 3 |
| 2 | 0 | 7 |
| 3 | 1 | 2 |

# Pandas

**Structure of table**

As mentioned before, 80% of the data we will see are table so, it's crucial to know its structure deeply.




- Index is like an address, that’s how any data point across the DataFrame or series can be accessed.
- Columns also called as features or fields are a list of values for the specific thing we are measure. for example, in the snippet we have every value in the apples column represents number of apples at that time.
- Rows also called as observations or occurrences are representing every object in our data. for example, here we have 4 rows which means we have 4 different counts of apples and oranges.

# Pandas



**Reading Data using Pandas**

Pandas can read data from wide range of formats such as:

- CSV files
- Excel files
- JSON files
- SQL Tables
- Pickle files
- Parquet files

Now, and for the sake of analysis, we will read a CSV file that is containing top 1000 in the history of film making according to IMDB website.
Let's see how.

```
# read csv file
df_movies = pd.read_csv('../imdb_movies.csv')
# see the data
display(df_movies)
# print its type
display(type(df_movies))
```

# Pandas



| | names | date_x | score | genre | overview | crew | orig_title | status | orig_lang | budget_x | revenue | country |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | Creed III | 03/02/2023 | 73.0 | Drama, Action | After dominating the boxing world, Adonis Cree... | Michael B. Jordan, Adonis Creed, Tessa Thompso... | Creed III | Released | English | 75000000.0 | 2.716167e+08 | AU |
| 1 | Avatar: The Way of Water | 12/15/2022 | 78.0 | Science Fiction, Adventure, Action | Set more than a decade after the events of the... | Sam Worthington, Jake Sully, Zoe Saldaña, Neyt... | Avatar: The Way of Water | Released | English | 460000000.0 | 2.316795e+09 | AU |
| 2 | The Super Mario Bros. Movie | 04/05/2023 | 76.0 | Animation, Adventure, Family, Fantasy, Comedy | While working underground to fix a water main,... | Chris Pratt, Mario (voice), Anya Taylor-Joy, P... | The Super Mario Bros. Movie | Released | English | 100000000.0 | 7.244590e+08 | AU |
| 3 | Mummies | 01/05/2023 | 70.0 | Animation, Comedy, Family, Adventure, Fantasy | Through a series of unfortunate events, three ... | Óscar Barberán, Thut (voice), Ana Esther Albor... | Momias | Released | Spanish, Castilian | 12300000.0 | 3.420000e+07 | AU |

10178 rows × 12 columns

```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None
```

/usr/local/lib/python3.10/dist-packages/pandas/core/frame.py
Two-dimensional, size-mutable, potentially heterogeneous tabular data.

# Pandas



**Basic Attributes and Methods**

- **df.info():** method used for getting information about number of rows,columns, count of not null, memory usage.
- **df.head():** method that views first 5 rows of the dataframe. You can pass any number in the brackets that will represent any number of rows you want to view.
- **df.tail():** method that views last 5 rows of the dataframe. You can pass any number in the brackets that will represent any number of rows you want to view.
- **df.describe():** method that can view the summary statistics for numerical columns.
- **df.shape():** attribute that is use to find out the number of rows and columns (Can you remember the same method in numpy?)
- **df.columns():** attribute that is used to view the column names and it's an iterator by default.

# Pandas



**Example 1**: we want to view the last 2 rows of the df

```
df_movies.tail(2)
```

| | names | date_x | score | genre | overview | crew | orig_title | status | orig_lang | budget_x | revenue | country |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 10176 | Darkman II: The Return of Durant | 07/11/1995 | 55.0 | Action, Adventure, Science Fiction, Thriller, ... | Darkman and Durant return and they hate each o... | Larry Drake, Robert G. Durant, Arnold Vosloo, ... | Darkman II: The Return of Durant | Released | English | 116000000.0 | 475661306.0 | US |
| 10177 | The Swan Princess: A Royal Wedding | 07/20/2020 | 70.0 | Animation, Family, Fantasy | Princess Odette and Prince Derek are going to ... | Nina Herzog, Princess Odette (voice), Yuri Low... | The Swan Princess: A Royal Wedding | Released | English | 92400000.0 | 539401838.6 | GB |

**Example 2**: we want to view our columns

```
df_movies.columns
```

```
Index(['names', 'date_x', 'score', 'genre', 'overview', 'crew', 'orig_title',
       'status', 'orig_lang', 'budget_x', 'revenue', 'country'],
      dtype='object')
```

# Pandas



- As you can see the df.describe() is calculating some summary statistics. Such as count, mean, std, min, max and the Percentile

Now, lets see the summary statistics using `df.describe()`

```
[10] df_movies.describe()
```

| | score | budget_x | revenue |
|---|---|---|---|
| count | 10178.000000 | 1.017800e+04 | 1.017800e+04 |
| mean | 63.497052 | 6.488238e+07 | 2.531401e+08 |
| std | 13.537012 | 5.707565e+07 | 2.777880e+08 |
| min | 0.000000 | 1.000000e+00 | 0.000000e+00 |
| 25% | 59.000000 | 1.500000e+07 | 2.858898e+07 |
| 50% | 65.000000 | 5.000000e+07 | 1.529349e+08 |
| 75% | 71.000000 | 1.050000e+08 | 4.178021e+08 |
| max | 100.000000 | 4.600000e+08 | 2.923706e+09 |

# Pandas



- For finding the number of Null values in the dataset we can use isnull() Function

```
# number of all nulls in every column summed
df_movies.isnull().sum()
```

```
names          0
date_x         0
score          0
genre         85
overview       0
crew          56
orig_title     0
status         0
orig_lang      0
budget_x       0
revenue        0
country        0
dtype: int64
```

# Pandas

Some column names in the dataset may case some confusing such as orig_title and orig_lang. We will try to rename them to Movie_Title and Movie_Language

- df.rename(): method for renaming columns it takes the columns in form of a dict where the old names are keys and the new names are values. Also, it takes an argument called inplace when its set to true it means that the current changes will be committed to the current dataframe directly.

```
# rename() :: method ---> for renaming column names
df_movies.rename(columns = {'orig_title' : 'Movie_Title', 'orig_lang': 'Movie_Language'},
                 inplace = True)
# check the new column names
df_movies.columns
```

```
Index(['names', 'date_x', 'score', 'genre', 'overview', 'crew', 'Movie_Title',
       'status', 'Movie_Language', 'budget_x', 'revenue', 'country'],
      dtype='object')
```

# Pandas

**Duplicate Rows Handling**:

1st: Duplicate the dataset

```
df_temp = pd.concat([df_movies, df_movies], axis = 0)
```

2nd view the dataset sorted

```
# Let's view df_temp sorted
df_temp.sort_values(by='movie_title', axis=0, inplace=True)
df_temp.head()
```

| | names | date_x | score | genre | overview | crew | movie_title | status | movie_language | budget_x | revenue | country |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1684 | #Alive | 06/24/2020 | 73.0 | Horror, Action, Adventure, Thriller | As a grisly virus rampages a city, a lone man ... | Yoo Ah-in, Oh Joon-woo, Park Shin-hye, Kim Yoo... | #살아있다 | Released | Korean | 6300000.0 | 13416285.0 | KR |
| 1684 | #Alive | 06/24/2020 | 73.0 | Horror, Action, Adventure, Thriller | As a grisly virus rampages a city, a lone man ... | Yoo Ah-in, Oh Joon-woo, Park Shin-hye, Kim Yoo... | #살아있다 | Released | Korean | 6300000.0 | 13416285.0 | KR |
| 10116 | The Beyond | 04/29/1981 | 69.0 | Horror | A young woman inherits an old hotel in Louisia... | Catriona MacColl, Liza Merril, David Warbeck, ... | ...E tu vivrai nel terrore! L'aldilà | Released | Italian | 101400000.0 | 531683245.8 | IT |
| 10116 | The Beyond | 04/29/1981 | 69.0 | Horror | A young woman inherits an old hotel in | Catriona MacColl, Liza Merril, David Warbeck | ...E tu vivrai nel terrore! L'aldilà | Released | Italian | 101400000.0 | 531683245.8 | IT |

# Pandas



**Removing Duplicates:**

ow, we will try removing duplicates using the following function

df.drop_duplicates(subset= [sequence of columns], keep='first/last/False', inplace=True/False)

```
* 'first' ---> to keep only the first occurace of the row
* 'last'  ---> to keep the last occurance of the row
* 'False' ---> to delete any duplicates with any number of occurances
```

# Pandas



Now we use function **drop_duplicates** and then check the shape of data frame again

```
# Now we will try to drop the full-row duplicates
df_temp.drop_duplicates(inplace=True)
# check the shape
df_temp.shape
```

```
(10178, 12)
```

# Pandas



Now, we'll remove any rows that have any duplicates. that's mean the remaining number of rows will be 0

```
df_temp.drop_duplicates(inplace=True, keep=False)
# check the shape
df_temp.shape
```

```
(0, 12)
```

# Pandas



**Missing Values Handling:**

Missing values are a common thing to check when exploring any data frame and they are some handling techniques for them such as the following:

- **Dropping Null Values**: When having a relatively small portion of rows with Null values we can simply drop them.
  - df.dropna(axis=1/0, inplace=True/False): This method will remove any rows with missing values.

# Pandas



**Missing Values Handling:**

- **Imputation**: Sometimes when the portion of Null values is slightly bigger, we can choose some values to add in those Nulls such as the mean or median value for numerical columns and may be the most or least frequent category for object columns. Sometimes the imputation will be in a systematic way such as the example we will touch now.
  - df.fillna(value, method='ffill/bfill', inplace='True/False'): This method will impute any values to the dataframe in place of Null values.

# Pandas

```
# lets use dropna to remove all rows with Null values
df_movies.dropna(axis = 0 , inplace=True )
#let's check if there are any null values
df_movies.isnull().sum()
```

```
names             0
date_x            0
score             0
genre             0
overview          0
crew              0
movie_title       0
status            0
movie_language    0
budget_x          0
revenue           0
country           0
dtype: int64
```

# What is Matplotlib?

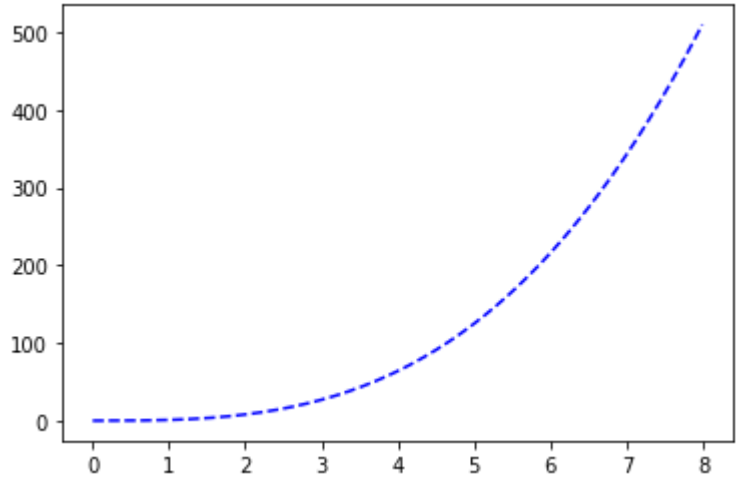
A Python programming language library used for plotting and visualizing data. It is the most prominent of all Python visualization packages and offers a wide range of functions. It can produce high-quality figures in various formats, such as PDF, SVG, JPG, PNG, BMP, and GIF.

# Functions to different plots

| Type of Plot | Function |
|---|---|
| Line Plot (Default) | `plt.plot()` |
| Vertical Bar Chart | `plt.bar()` |
| Horizontal Bar Chart | `plt.barh()` |
| Error Bar Chart | `plt.errorbar()` |
| Histogram | `plt.hist()` |
| Box Plot | `plt.box()` |
| Area Plot | `plt.area()` |
| Scatter Plot | `plt.scatter()` |
| Pie Plot | `plt.pie()` |
| Hexagonal Bins Plot | `plt.hexbin()` |

# Basic Plot Types in Matplotlib

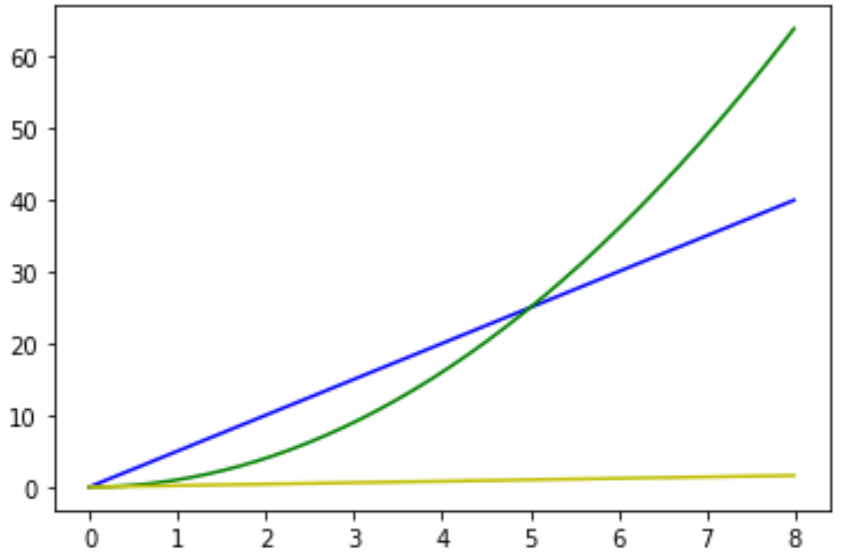
```
# import packages
import pandas as pd
import numpy as np
# setting random seed
np.random.seed(101)
# import matplotlib
import matplotlib.pyplot as plt

# create random array
rnd_arr = np.arange(0, 8, 0.01)
# plot function
#plt.plot(rnd_arr, [i**3 for i in rnd_arr ], 'b--')
plt.plot(rnd_arr, np.power(rnd_arr,3), 'b--')     #take the randome numbers we generated then find its power 3
# show the plot
plt.show()
```



# Basic Plot Types in Matplotlib

## 1. Multiline Plot

```
plt.plot(rnd_arr, [i*5 for i in rnd_arr], 'b-')
plt.plot(rnd_arr, [i**2 for i in rnd_arr], 'g-')
plt.plot(rnd_arr, [i/5 for i in rnd_arr], 'y-')
# show
plt.show()
```

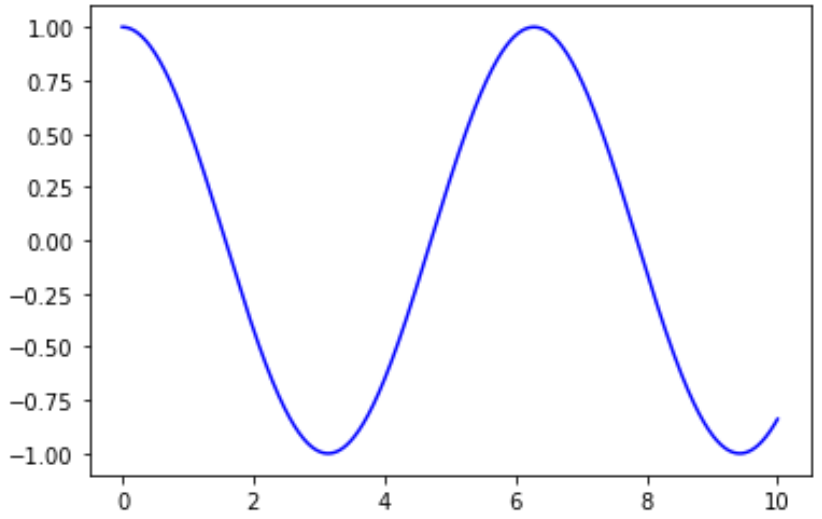


# Basic Plot Types in Matplotlib

## 2. Line Plot

A line plot uses line segments to connect data points. Line plots are one of the most commonly used types of plots. These are especially useful in situations that show changes or other variables that occur over a period of time.

```
# create rnd_x :: 1000 random point between 0 and 1
rnd_x = np.linspace(0, 10, 1000)
# create rnd_y :: cos value of each point in x
rnd_y = np.cos(rnd_x)
# plot
plt.plot(rnd_x, rnd_y, 'b-');
```



# Basic Plot Types in Matplotlib

## 3. Bar Plot

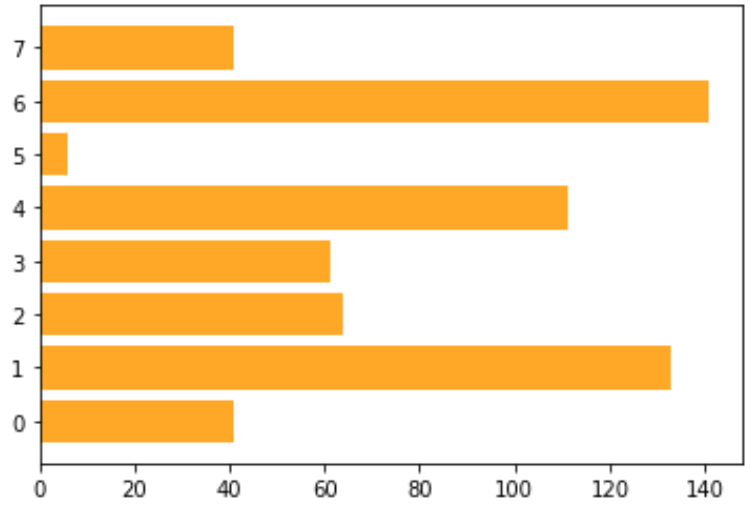
Bar graphs are ideal for demonstrating segments of information, especially when comparing different categories or discrete variables. They are particularly useful for comparing age groups, classes, schools and the like, as long as there are not too many to compare. Additionally, bar graphs are well-suited for displaying time series data due to the limited space on the x-axis that is ideal for representing years, minutes, hours or months.

# Basic Plot Types in Matplotlib

## 3. Bar Plot (Horizontal)

Horizontal Bar Chart is very similar to normal Bar Chart except that a horizontal bar chart is a great option for long category names. This is because there is space left of the chart for horizontal alignment of the axis labels.

```
# create random list of 8 numbers
rnd_lst = list(np.random.randint(1,150,8))
# plot bar graph
plt.barh(range(len(rnd_lst)), rnd_lst, color = '#FFA726')
# show
plt.show()
```

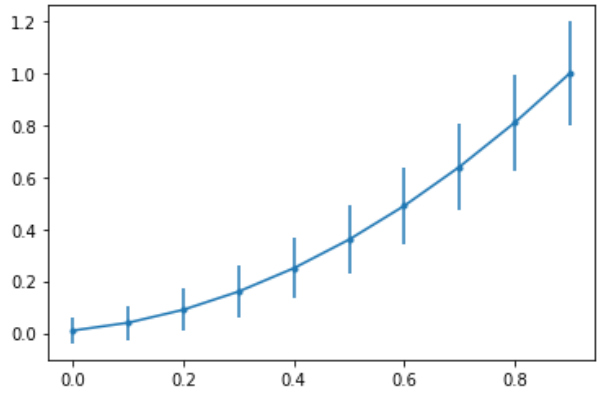


# Basic Plot Types in Matplotlib

## 4. Error Bar Chart

Error bars are useful for showing estimated error or uncertainty and giving a general idea of how accurate a measurement is. It is done by drawing. To visualize this information, error bars work by drawing lines extending from the center of the plotted data points or the edge of the bar chart. Short error bars indicate that the values are concentrated and the plotted mean is more likely, while longer error bars indicate that the values are more scattered and less reliable.

```
# defining our function
rndx = np.arange(10)/10
rndy = (rndx + 0.1)**2

# defining our error
y_error = np.linspace(0.05, 0.2, 10)

# plot error bars
plt.errorbar(rndx, rndy, yerr = y_error, fmt = '.-');
```

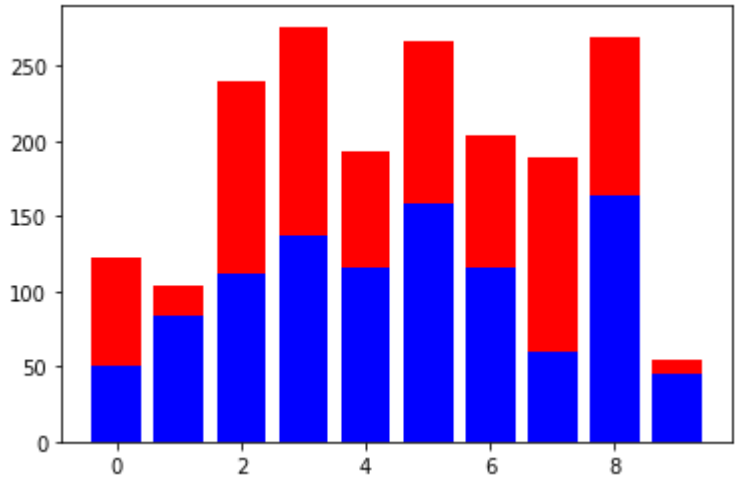


# Basic Plot Types in Matplotlib

## 5. Stacked Bar Chart

A stacked chart is a type of bar chart that shows how two or more variables compare over time. Also called a stacked bar or column chart, it looks like a series of columns or bars that are stacked on top of each other. Stacked charts are very useful for comparisons when used correctly. They are designed to compare total values across categories.

```
# lets create two random lists :: will be stacked on top of each other
rnd_a = list(np.random.randint(1,200,10))
rnd_b = list(np.random.randint(1,200,10))
# use simple range function for marking x steps
x_steps = range(10)

# for a stacked bar we will use two separte plt.bar() before calling plt.show()
plt.bar(x_steps, rnd_a, color = 'b')
plt.bar(x_steps, rnd_b, color = 'r', bottom = rnd_a)

plt.show()
```

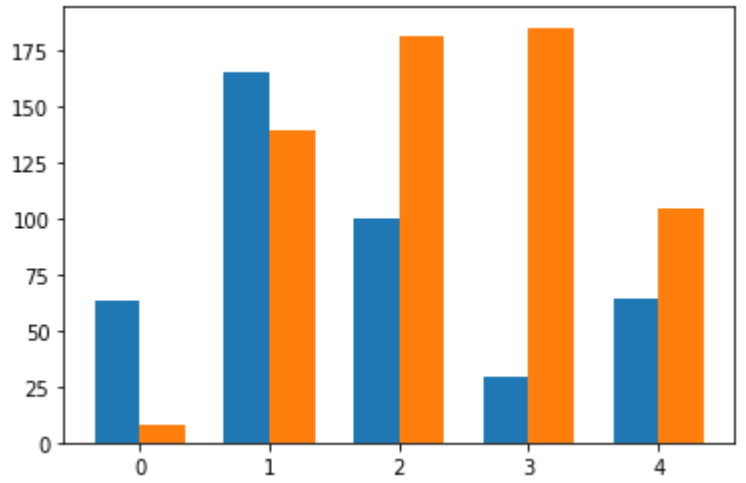


# Basic Plot Types in Matplotlib

## 6. Grouped Bar Chart

Grouped Bar Chart is trying to solve the main drawback of Stacked Bar chart by having a separate bar for every sub-group (sub-category) while having the related sub-groups aligned horizontally.

```
# create two random lists that are reresents the amount bought of product a, and b respectively
prod_a = list(np.random.randint(1,200,5))
prod_b = list(np.random.randint(1,200,5))

# the label locations
x = np.arange(len(prod_a))
width = 0.35  # the width of the bars

# create two
plt.bar(x - width/2, prod_a, width, label='Men')
plt.bar(x + width/2, prod_b, width, label='Women')
# show
plt.show()
```

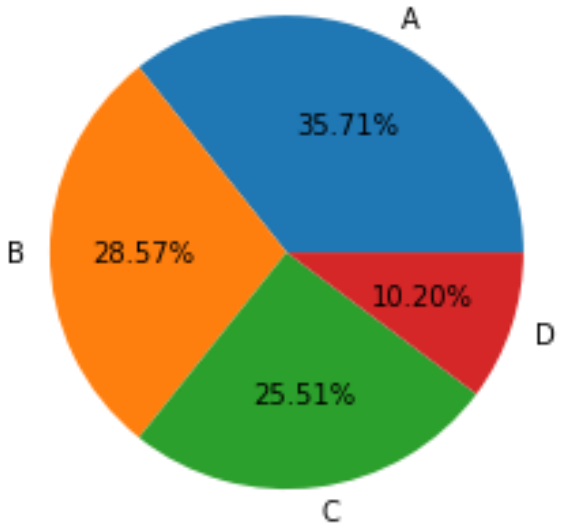


# Basic Plot Types in Matplotlib

## 7. Pie Plot

Pie plots can only be used when the individual parts add up to a meaningful whole and are used to visualize how each part contributes to the whole.

```
# random array
rndnums = np.array([35, 28, 25, 10])
# labels
labels = ['A', 'B', 'C', 'D']
# create pie plot
plt.pie(rndnums, labels = labels, autopct = '%.2f%%')
# show
plt.show()
```

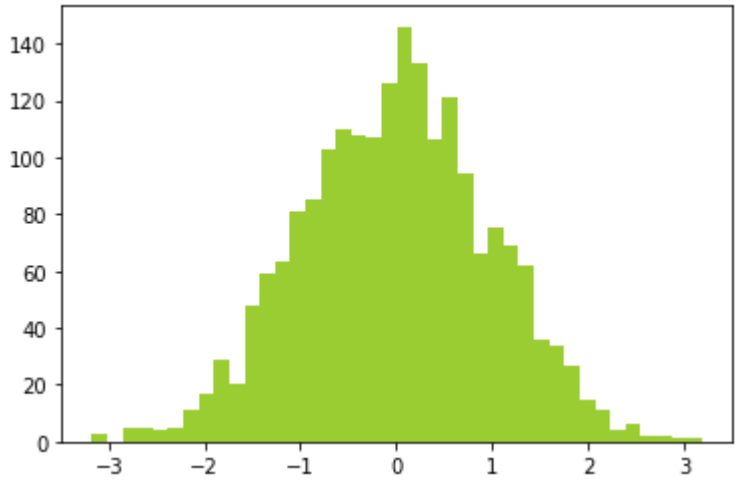


# Basic Plot Types in Matplotlib

## 8. Histogram

Histograms or frequency distribution plots indicates how often each distinct value occurs in a data set. Histograms are the most commonly used charts for showing frequency distributions. Although they are very similar to bar charts, there are important differences between the two. Histograms visualize quantitative data or numerical data, whereas bar charts display categorical variables.

```
# create random normal distrbution
rndx = np.random.normal(size = 2000)
# create histogram
plt.hist(rndx, bins=40, color='yellowgreen')
# show
plt.show()
```

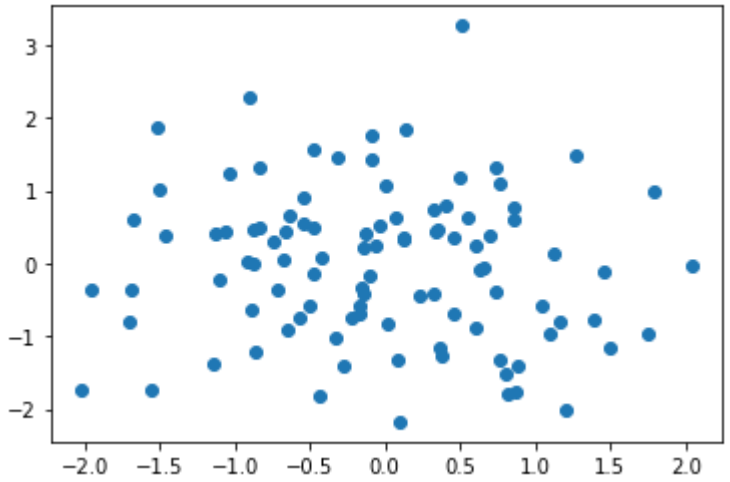


# Basic Plot Types in Matplotlib

## 9. Scatter Plot

Scatter Plot is a type of graph that shows the relationship between two numerical values by plotting their individual points on a graph. Each point is represented by a dot.

```
# generate random data
x = np.random.normal(size=100)
y = np.random.normal(size=100)

# plot the data
plt.scatter(x, y)
# show the plot
plt.show()
```



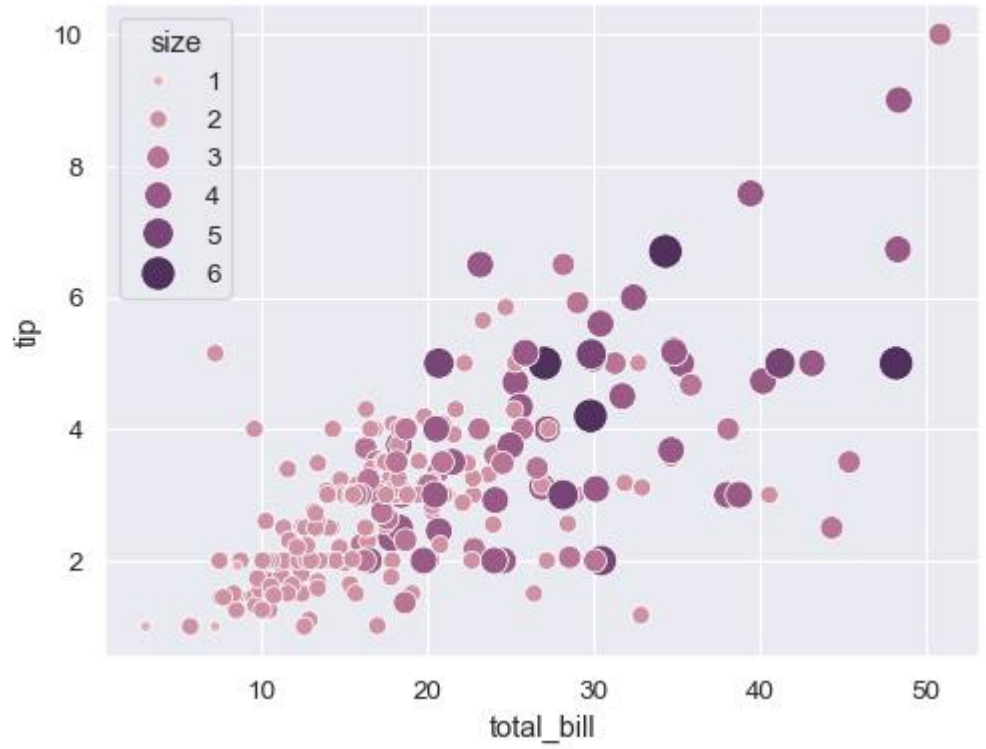
# What is Seaborn?

Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. It is particularly useful for creating complex plots with minimal code.

# Basic Plot Types in Seaborn

1. Scatter Plot

Scatter plots are used to show the relationship between two numerical variables. Here's how to create a scatter plot

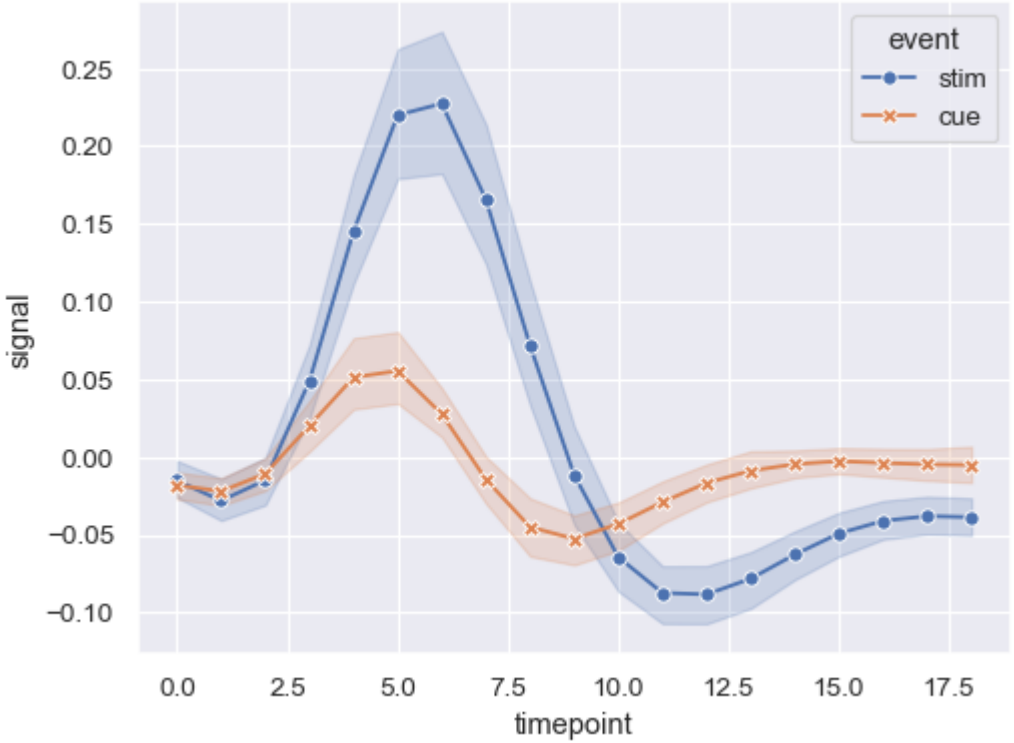



```python
# Import necessary libraries
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Creating sample data
np.random.seed(42)  # For reproducibility
data = pd.DataFrame({
    'x': np.random.rand(100),  # 100 random numbers between 0 and 1 for 'x'
    'y': np.random.rand(100)   # 100 random numbers between 0 and 1 for 'y'
})

# Creating a scatter plot
sns.scatterplot(x='x', y='y', data=data)
plt.title('Scatter Plot of X vs Y')  # Adding a title
plt.xlabel('X-axis Label')           # Adding an x-axis label
plt.ylabel('Y-axis Label')           # Adding a y-axis label
plt.show()                           # Display the plot
```

# Basic Plot Types in Seaborn

## 2. Line Plot

Line plots are used to show trends over time or other continuous variables

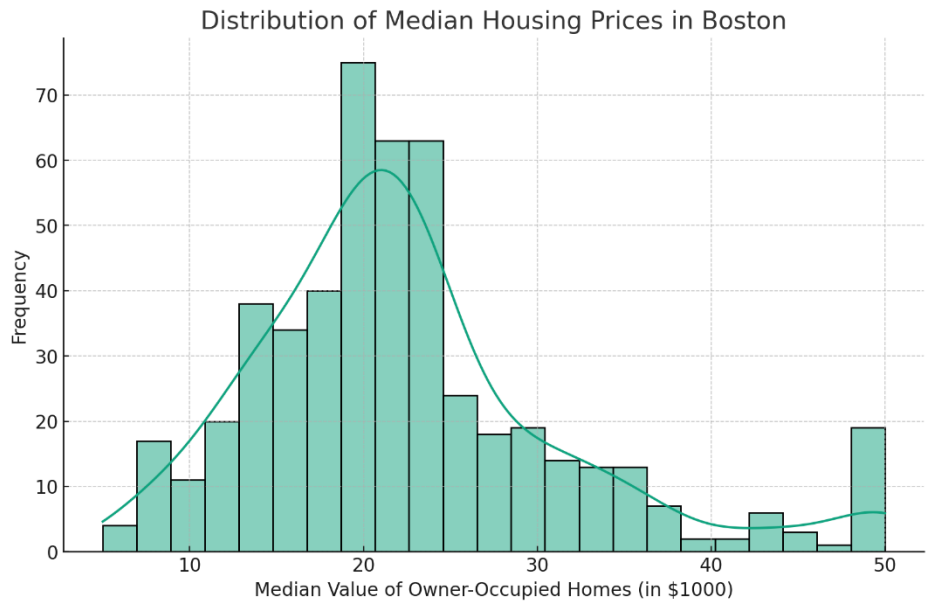



```python
# Sample data
data = np.random.randn(1000)  # 1000 random numbers from a normal distribution

# Creating a histogram
sns.histplot(data, bins=30, kde=True)
plt.title('Histogram with KDE')  # Adding a title
plt.xlabel('Value')              # Adding an x-axis label
plt.ylabel('Frequency')          # Adding a y-axis label
plt.show()                       # Display the plot
```

# Basic Plot Types in Seaborn

## 3. Histogram

Histograms show the distribution of a single numerical variable

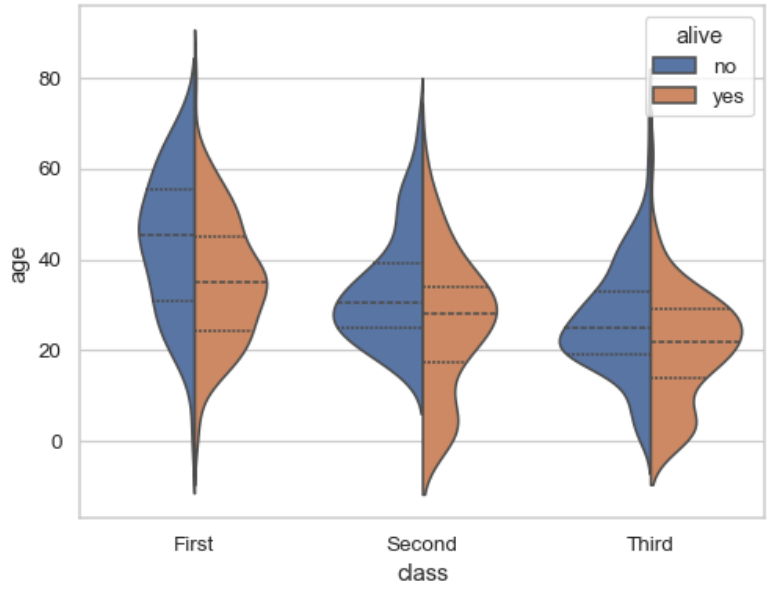



```
# Sample data
categories = ['A', 'B', 'C', 'D']
values = [10, 23, 17, 5]

# Creating a bar plot
sns.barplot(x=categories, y=values)
plt.title('Bar Plot of Categories')  # Adding a title
plt.xlabel('Category')               # Adding an x-axis label
plt.ylabel('Value')                  # Adding a y-axis label
plt.show()                           # Display the plot
```

# Basic Plot Types in Seaborn

## 4. Violin Plot

Violin plots combine aspects of box plots and density plots. They show the distribution of the data across different categories

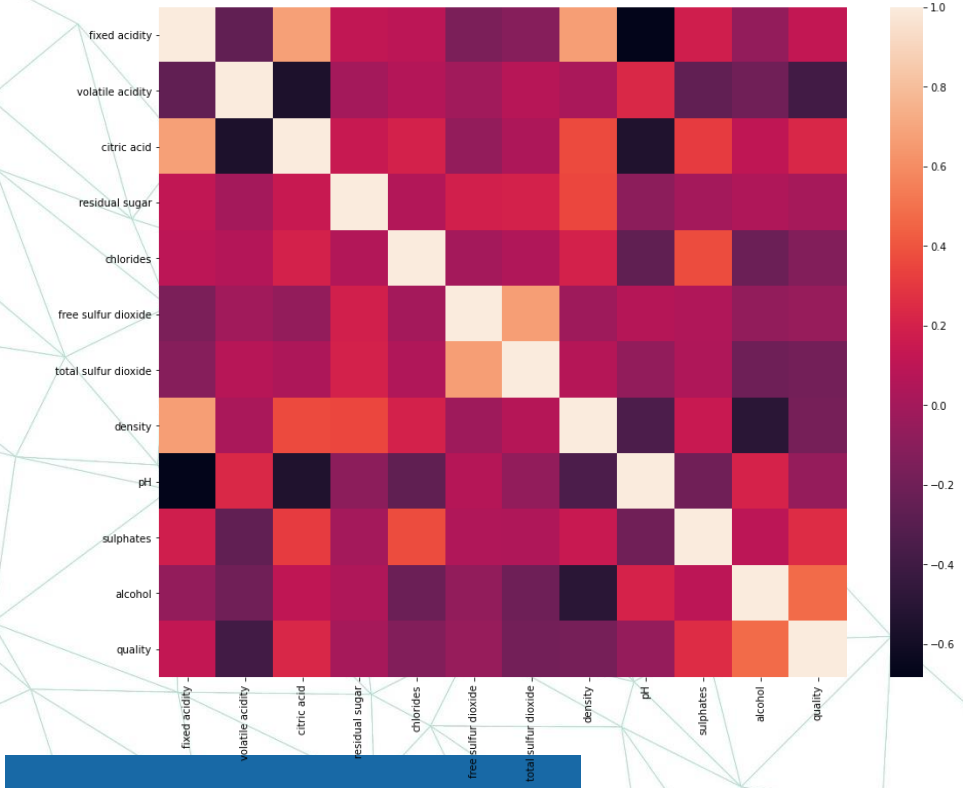



```
# Creating a violin plot
sns.violinplot(x='category', y='value', data=data)
plt.title('Violin Plot of Values by Category')  # Adding a title
plt.xlabel('Category')                          # Adding an x-axis label
plt.ylabel('Value')                             # Adding a y-axis label
plt.show()                                      # Display the plot
```

# Basic Plot Types in Seaborn

## 5. Heatmap

Heatmaps display matrix-like data where each cell is colored according to its value. They are useful for visualizing the correlation between variables or any kind of matrix data



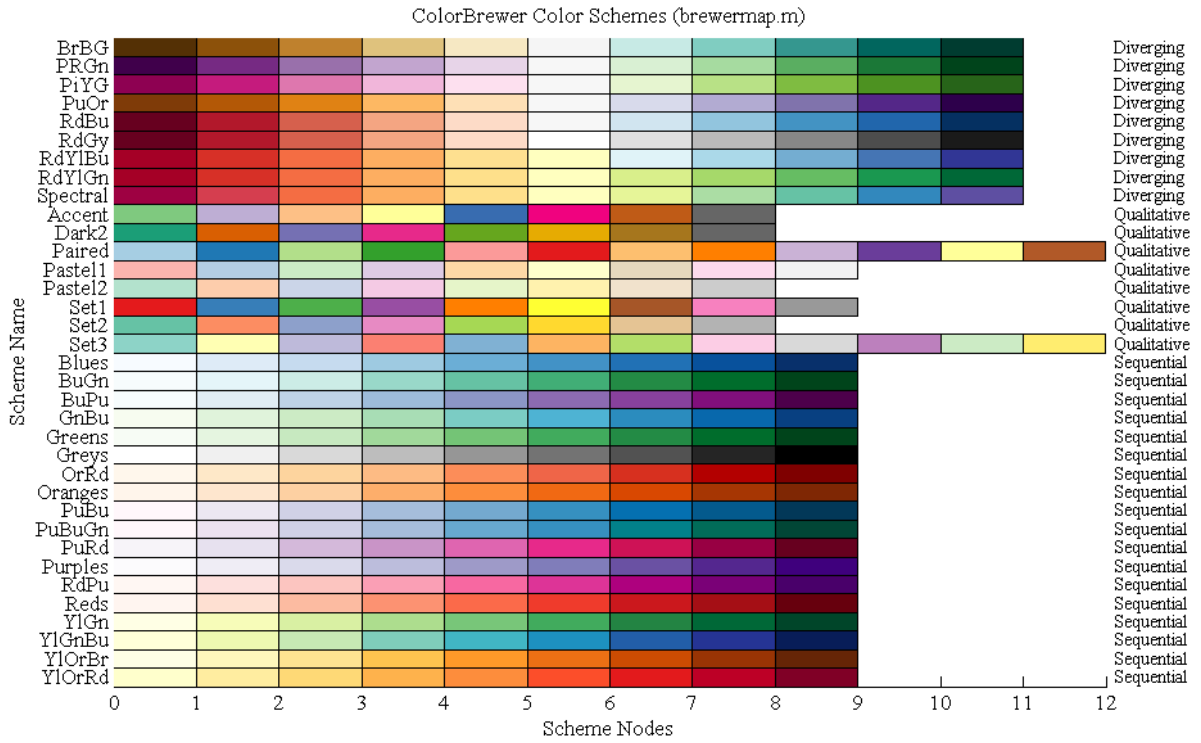

```
data = np.random.rand(10, 12)  # 10x12 matrix of random numbers

# Creating a heatmap
sns.heatmap(data, annot=True, cmap='coolwarm')
plt.title('Heatmap of Random Data')  # Adding a title
plt.show()                           # Display the plot
```

# Customizing Seaborn Plots

- Changing Color Palettes

  Seaborn offers several built-in color palettes and allows custom color schemes.

**sns.set_palette('pastel')**

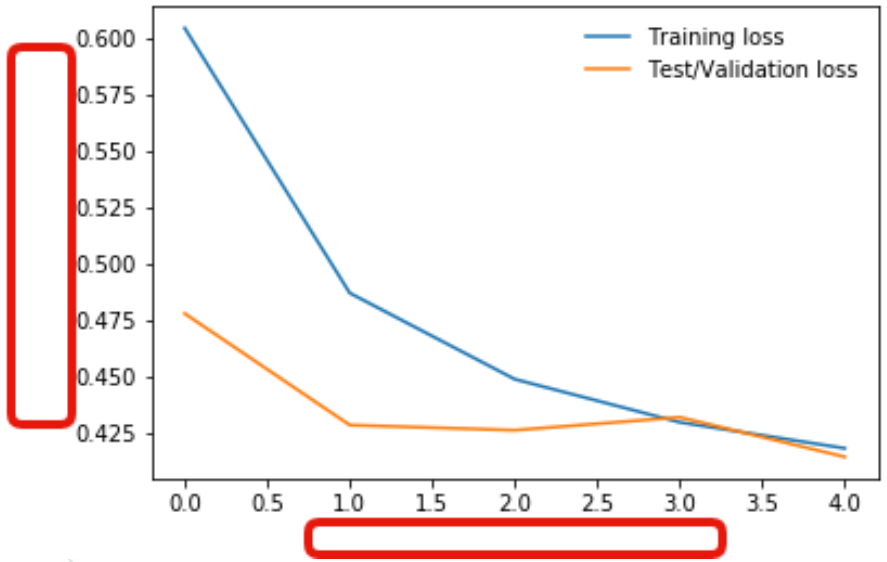


# Customizing Seaborn Plots

- Adding Titles and Labels

  You can add titles and labels to your plots using Matplotlib's functions.

**plt.title('Bar Plot of Categories’)**
**plt.xlabel('Category**
**plt.ylabel('Value')**

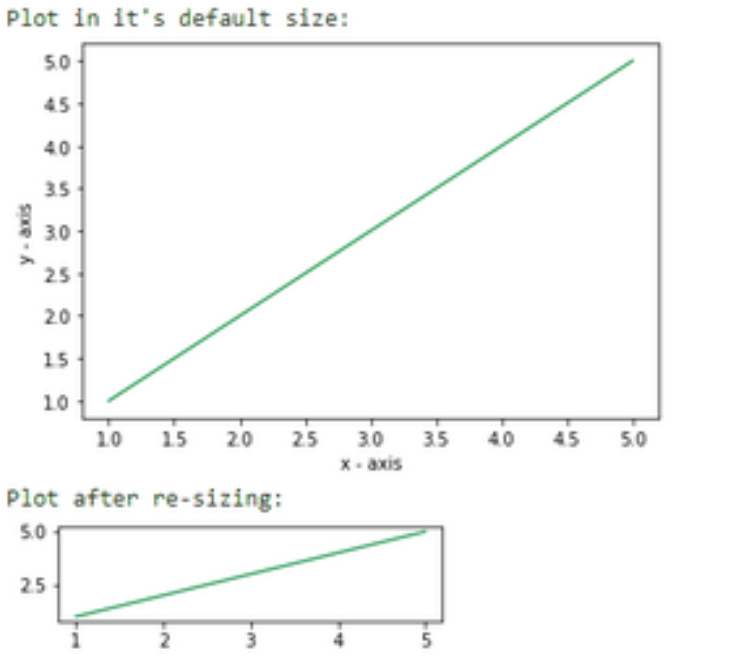


# Customizing Seaborn Plots

- Adjusting Plot Size

  The size of the plots can be adjusted using Matplotlib's **plt.figure()** function .

**plt.figure(figsize=(10, 6))**



# Customizing Seaborn Plots

- Saving Plots

  You can save your Seaborn plots to files using Matplotlib's **plt.savefig()** function.

  **plt.savefig('scatter_plot.png')**

# Data Visualization with NumPy and Matplotlib

- **Plotting Basics:** Use Matplotlib to create plots that give insights into the data.

```
plt.plot(arr)
plt.show()
```

- **Histograms:** Show the distribution of data.

```
plt.hist(arr, bins=10)
plt.show()
```

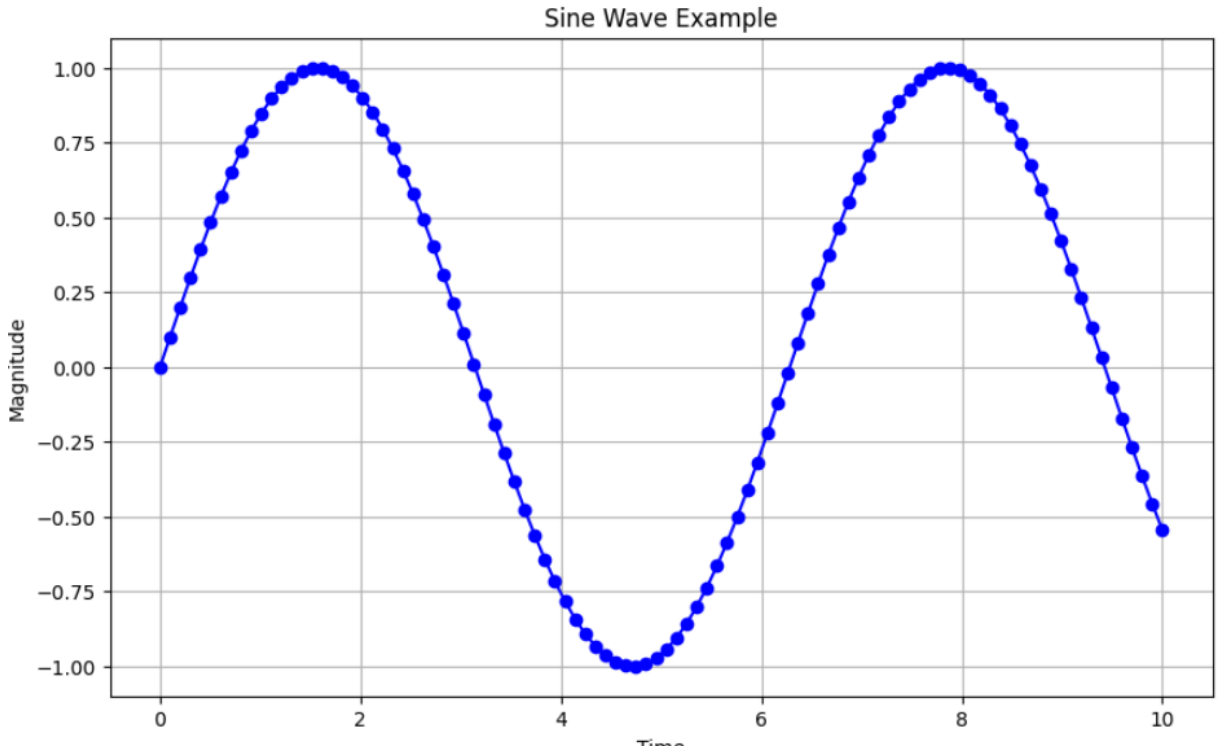
- **Scatter Plots:** Visualize relationships between two variables.

```
plt.scatter(arr1, arr2)
plt.show()
```

# Data Visualization with NumPy and Matplotlib

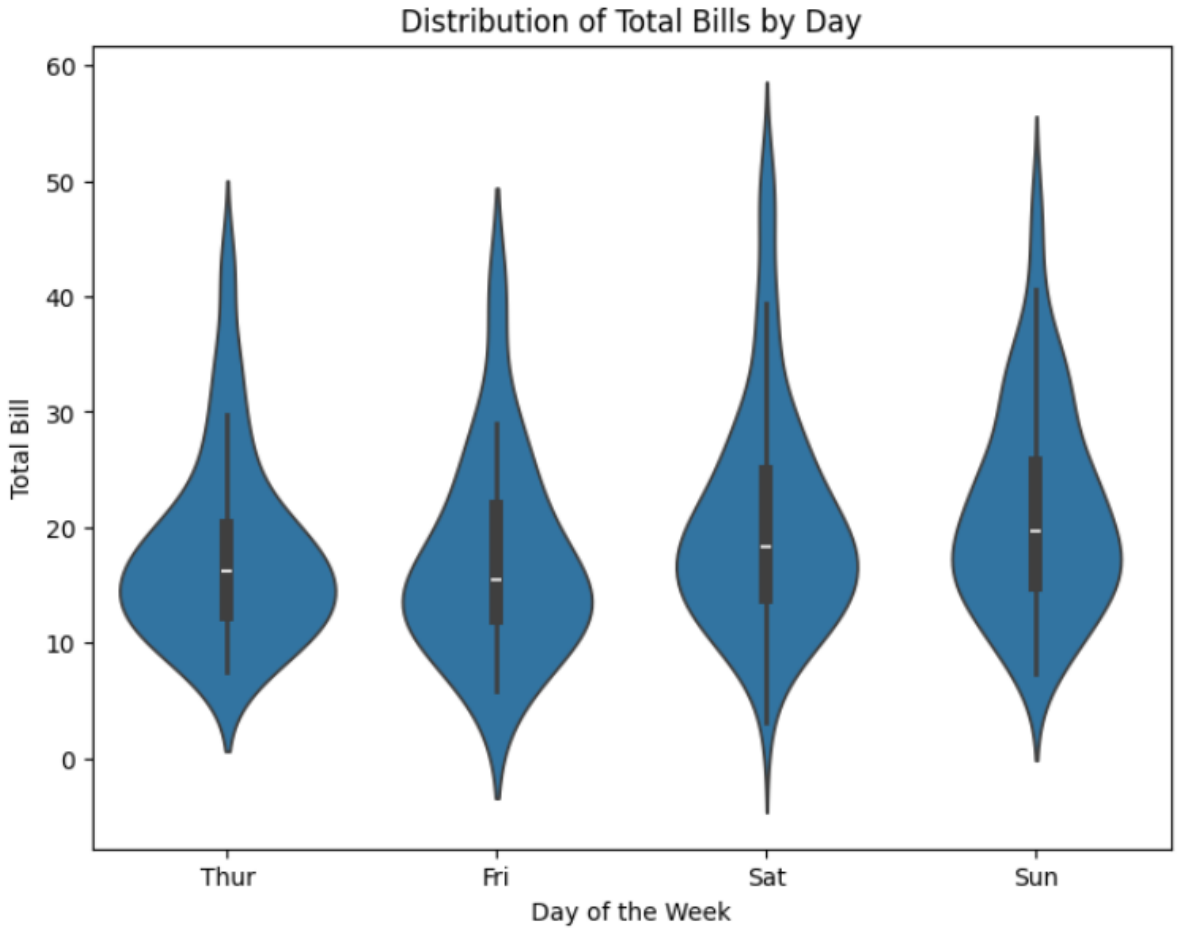


```python
import matplotlib.pyplot as plt
import numpy as np

# Generating sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Creating a line plot
plt.figure(figsize=(10,6))
plt.plot(x, y, marker='o', color='blue')
plt.title('Sine Wave Example')
plt.xlabel('Time')
plt.ylabel('Magnitude')
plt.grid(True)
plt.show()
```



# Advanced Visualization with Seaborn and Pandas

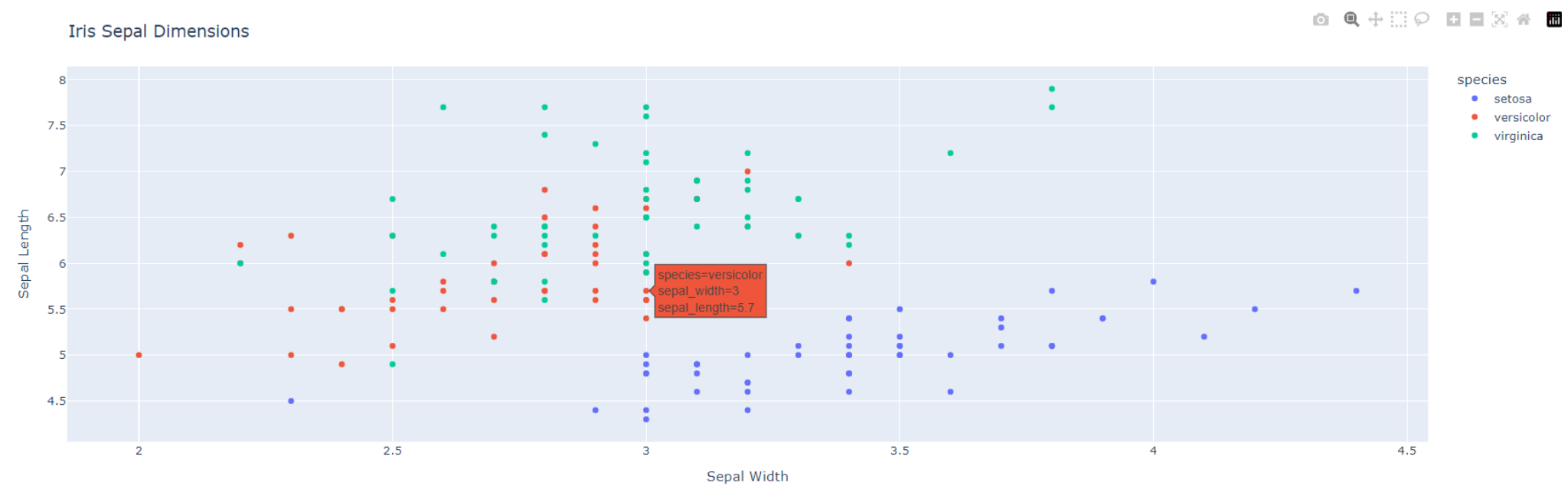
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
# Loading dataset
tips = sns.load_dataset('tips')

# Creating a violin plot
plt.figure(figsize=(8,6))
sns.violinplot(x='day', y='total_bill', data=tips)
plt.title('Distribution of Total Bills by Day')
plt.xlabel('Day of the Week')
plt.ylabel('Total Bill')
plt.show()
```



# Interactive Visualizations



```
# Importing libraries
import plotly.express as px

# Loading data
iris = px.data.iris()

# Creating an interactive scatter plot
fig = px.scatter(iris, x='sepal_width', y='sepal_length', color='species')
fig.update_layout(title='Iris Sepal Dimensions', xaxis_title='Sepal Width', yaxis_title='Sepal Length')
fig.show()
```



# Visualization Best Practices

- **Choose the Right Chart Type:** Match the chart type with the story you want to tell.
- **Keep It Simple:** Avoid clutter and focus on the data.
- **Consistent Style:** Use consistent colors and fonts to make your chart cohesive and professional.
- **Accessible:** Ensure your visuals are accessible to all audiences, including those with color vision deficiencies.

# Any Questions ?

# Thank You